

app\main.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      main.py
4  @brief     Système de mesure et simulation de température.
5  @details   Ce module exécute des cycles de mesure ADC, convertit les valeurs
6             en résistances puis en températures, récupère des paramètres via
7             une API distante, calcule une température simulée, convertit
8             cette température en résistance cible et applique la simulation
9             via un multiplexeur matériel. Le module gère également un petit
10            serveur web optionnel et un indicateur LED.
11  @author    Léo Mendes
12  @project   2510_RegulationsChauffage
13  @Mandant   Oblo_solution
14  @date      2025-09-18
15  @version   1.0.0
16  @copyright Copyright (c) 2025
17  """
18
19  from __future__ import annotations
20
21  # Opérations système et manipulation de chemins
22  import os
23  # Accès à la table des chemins d'import et arguments
24  import sys
25
26  # Calculer le répertoire courant du module et le répertoire source parent
27  # Dossier contenant ce fichier
28  CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
29  # Dossier parent, racine du paquet local
30  SRC_DIR = os.path.dirname(CURRENT_DIR)
31  # S'assurer que le dossier source est dans sys.path pour permettre les imports relatifs locaux
32  if SRC_DIR not in sys.path:
33      sys.path.append(SRC_DIR)
34
35  # Fonctions temporelles (sleep, timestamps)
36  import time
37  # Parsing des arguments de la ligne de commande
38  import argparse
39  # Fonctions mathématiques utiles
40  import math
41  # Gestion multithread (input monitor, webui thread)
42  import threading
43  # Gestion des signaux système (Ctrl+C)
44  import signal
45  # Surveillance d'entrées/sockets si nécessaire
46  import select
47  # Client HTTP pour l'API distante
48  import requests
49  # Sérialisation/désérialisation JSON
50  import json
51  # Manipulation de dates et intervalles
52  from datetime import datetime, timedelta
53  # Abstraction GPIO (bibliothèque periphery)
54  from periphery import GPIO
55  # Constantes matérielles pour le CM5
56  from pins_cm5 import GPIO_CHIP_PATH, FRONT_LED, CMD_RELAY
57  # Utilitaire de chargement de configuration JSON
58  from app.config_loader import load_config
59  # Profils de capteurs et tables associées
60  from app.sensor_profiles import get_profile, SENSOR_TABLES
61  # Pilote du multiplexeur matériel TMUX1204
62  from hw_tmux1204 import Tmux1204
63  # Pilote ADC ADS124S08
64  from app.adc_ads124s08 import Ads124s08, request_stop as request_adc_stop, reset_stop_flag as reset_adc_stop_flag, is_stop_requested as is_adc_stop_requested
65  # Conversion T -> R pour sondes
66  from app.temperature_conversion import convert_temperature_to_resistance
67  # Client API (fonctions d'accès réseau)
68  from api_client import ApiClient
69  # Moteur de simulation de température
70  from temperature_simulation import run_temperature_simulation
71  # Simulation/application de résistance via MUX
72  from resistance_simulation import ResistanceSimulator
73  # Création/gestion de l'indicateur LED frontal
74  from front_led_management import create_led_indicator
75  # Test des entrées 24V (contacts secs)
76  from mesure_24v_dry_contact import test_24v_dry_contact, get_dry_contact_status
77
78  def start_webui_server():
79      """
80      @brief   Lance le serveur web local dans un thread séparé.
81      @details Import dynamique du module `webui.webui_server` et démarrage de
82              l'application Flask/WSGI. Les exceptions sont interceptées et
83              converties en messages sur la sortie standard pour ne pas

```

```

84         interrompre le thread principal.
85     @return None
86     """
87     try:
88         # Import retardé pour permettre l'exécution sans webui
89         from webui import webui_server
90         # Démarrage du serveur Flask avec les paramètres spécifiés
91         webui_server.app.run(host="192.168.1.109", port=8080, debug=False, use_reloader=False)
92     except ImportError:
93         # Gestion du cas où le module webui_server n'est pas disponible
94         print("[WEBUI] Module webui_server non trouvé")
95     except Exception as e:
96         # Gestion des autres erreurs du serveur web
97         print(f"[WEBUI] Erreur serveur web: {e}")
98
99 # -----
100 # Variables globales pour le contrôle du séquenceur
101 # -----
102
103 # Drapeau global indiquant que le séquenceur tourne
104 sequencer_running = False
105 # Événement pour demander l'arrêt propre
106 stop_event = threading.Event()
107 # Événement pour forcer une exécution manuelle
108 force_sequence = threading.Event()
109
110 # Instance globale de gestion LED
111 # Instance globale du gestionnaire LED (ou None si non disponible)
112 led_indicator = None
113
114 def save_last_state(channel=None, resistance_ohm=None, temperature_c=None, temperature_sim_c=None, error=None):
115     """
116     @brief Sauvegarde l'état courant et historique par canal dans un fichier JSON.
117     @details Crée le dossier `state` si nécessaire, lit le fichier `last_state.json`
118             existant, met à jour les informations du canal fourni et écrit le
119             document au format JSON. Les erreurs internes sont capturées et
120             affichées sur la sortie standard.
121
122     @param channel Numéro du canal (ou None).
123     @param resistance_ohm Résistance mesurée (ou None).
124     @param temperature_c Température mesurée [°C] (ou None).
125     @param temperature_sim_c Température simulée [°C] (ou None).
126     @param error Message d'erreur s'il y en a un (ou None).
127     @return None
128     """
129     try:
130         # Répertoire de stockage
131         state_dir = os.path.join(os.path.dirname(CURRENT_DIR), "state")
132         # Crée le répertoire si absent
133         os.makedirs(state_dir, exist_ok=True)
134         # Chemin complet du fichier
135         state_path = os.path.join(state_dir, "last_state.json")
136
137         # Charger l'état existant depuis le fichier JSON
138         try:
139             # Lecture du fichier JSON existant
140             with open(state_path, "r", encoding="utf-8") as f:
141                 existing_data = json.load(f)
142         except (FileNotFoundError, json.JSONDecodeError):
143             # Structure par défaut si inexistant
144             existing_data = {"channels": {}}
145
146         # S'assurer de la présence de la clé 'channels'
147         if "channels" not in existing_data:
148             existing_data["channels"] = {}
149
150         # Mettre à jour les données du canal spécifique si fourni
151         if channel is not None:
152             existing_data["channels"][str(channel)] = {
153                 "timestamp": datetime.now().isoformat(),
154                 "resistance_ohm": resistance_ohm,
155                 "temperature_c": temperature_c,
156                 "temperature_sim_c": temperature_sim_c,
157                 "error": error,
158             }
159         # Affichage de confirmation de sauvegarde
160         print(f"[STATE] Canal {channel} sauvegardé: {temperature_c}°C")
161
162         # Mettre à jour les méta-données globales (dernière mesure)
163         existing_data.update({
164             "last_timestamp": datetime.now().isoformat(),
165             "last_channel": channel,
166             "last_resistance_ohm": resistance_ohm,
167             "last_temperature_c": temperature_c,
168             "last_temperature_sim_c": temperature_sim_c,
169             "last_error": error,
170         })

```

```

171
172     # Écriture finale du fichier JSON
173     with open(state_path, "w", encoding="utf-8") as f:
174         json.dump(existing_data, f, ensure_ascii=False, indent=2)
175 except Exception as e:
176     # Gestion des erreurs de sauvegarde
177     print(f"[STATE] Erreur sauvegarde état: {e}")
178
179 def signal_handler(sig, frame):
180     """
181     @brief   Gestionnaire POSIX pour les signaux (Ctrl+C).
182     @details Met le drapeau `sequencer_running` à False et déclenche
183             l'événement `stop_event` pour demander un arrêt propre.
184             Demande aussi l'arrêt des opérations ADC en cours.
185     @param   sig   Signal reçu (int).
186     @param   frame Contexte d'exécution (non utilisé).
187     @return  None
188     """
189     global sequencer_running
190     # Affichage du message d'arrêt
191     print("\n[SEQUENCER] Arrêt demandé...")
192     # Arrêt du séquenceur
193     sequencer_running = False
194     # Déclenchement de l'événement d'arrêt
195     stop_event.set()
196     # Demande d'arrêt des opérations ADC
197     request_adc_stop()
198     print("[SEQUENCER] Arrêt des opérations ADC demandé...")
199
200 def input_monitor():
201     """
202     @brief   Thread d'écoute des commandes utilisateur depuis stdin.
203     @details Permet de forcer une séquence (touche Entrée ou 's') ou de
204             demander l'arrêt ('q'). Les exceptions d'E/S sont gérées pour
205             permettre la terminaison propre du thread.
206     @return  None
207     """
208     # Boucle de surveillance des entrées utilisateur
209     while sequencer_running and not stop_event.is_set():
210         try:
211             # Lit la ligne utilisateur
212             user_input = input()
213             # Vérification des commandes de déclenchement manuel
214             if user_input.strip().lower() in ['', 's', 'sequence', 'start']:
215                 print("[MANUAL] Séquence manuelle déclenchée")
216                 # Déclenchement manuel
217                 force_sequence.set()
218             # Vérification des commandes d'arrêt
219             elif user_input.strip().lower() in ['q', 'quit', 'exit']:
220                 print("[MANUAL] Arrêt demandé par l'utilisateur")
221                 # Simule Ctrl+C
222                 signal_handler(signal.SIGINT, None)
223                 break
224             except (EOFError, KeyboardInterrupt):
225                 break
226             except Exception:
227                 # Ignorer les erreurs d'entrée pour rester résilient
228                 pass
229
230 def show_error_and_continue(error_pattern, sleep_time=2):
231     """
232     @brief   Affiche un motif LED d'erreur pendant un temps donné puis
233             remet l'indicateur en mode 'sequence_running'.
234     @param   error_pattern Nom du motif LED à afficher.
235     @param   sleep_time    Durée d'affichage en secondes (par défaut 2s).
236     @return  None
237     """
238     global led_indicator
239     # Afficher l'erreur si l'indicateur LED est disponible
240     if led_indicator:
241         # Afficher l'erreur
242         led_indicator.set_pattern(error_pattern)
243     # Temporisation pour permettre la lecture
244     time.sleep(sleep_time)
245     # Revenir en mode séquence si l'indicateur LED est disponible
246     if led_indicator:
247         # Revenir en mode séquence
248         led_indicator.set_pattern('sequence_running')
249
250 def run_measurement_sequence(cfg, mac_address, adc, tmux):
251     """
252     @brief   Exécute une séquence complète de mesures pour tous les canaux
253             activés selon la configuration fournie.
254     @details Pour chaque canal actif : sélection du rref, configuration ADC,
255             lecture des valeurs, conversion en température, envoi API, calcul
256             de la température simulée et application via le simulateur de
257             résistance. Les motifs LED sont ajustés en fonction des erreurs.

```

```
258
259 @param cfg Dictionnaire de configuration chargé depuis JSON.
260 @param mac_address Adresse MAC ou identifiant pour les appels API.
261 @param adc Instance de l'ADC (Ads124s08).
262 @param tmux Instance du multiplexeur (Tmux1204).
263 @return bool True si la séquence est globalement considérée comme
264 réussie, False sinon.
265
266 """
267 global led_indicator
268 # Affichage du début de séquence avec horodatage
269 print(f"\n[SEQUENCER] Nouvelle séquence - {datetime.now().strftime('%H:%M:%S')}")
270
271 # Rechargement de la configuration depuis le fichier JSON
272 print(f"[CONFIG] Rechargement configuration...")
273 # Chemin vers le fichier de configuration
274 CONFIG_FILE = os.path.join(SRC_DIR, "config_module", "sensors.json")
275 try:
276     # Lecture et parsing du JSON - RECHARGEMENT COMPLET
277     cfg = load_config(CONFIG_FILE)
278     # Filtrage des canaux activés
279     enabled_channels = [ch for ch in cfg.get("channels", []) if ch.get("enabled")]
280     # Affichage du nombre de canaux actifs
281     print(f"[CONFIG] Config rechargée: {len(enabled_channels)} canaux actifs")
282     # Affichage du détail des canaux actifs
283     for ch_info in enabled_channels:
284         print(f"[CONFIG] Canal {ch_info['channel']}: {ch_info['sensor']}")
285
286     # Affichage des paramètres de temporisation rechargés
287     print(f"[CONFIG] Timeout ADC: {cfg.get('timeout_s', 180)}s")
288     print(f"[CONFIG] Délai inter-mesure: {cfg.get('inter_measure_sleep_s', 0.1)}s")
289     print(f"[CONFIG] Intervalle séquences: {cfg.get('sequence_interval_minutes', 5)} min")
290     print(f"[CONFIG] Mode automatique: {cfg.get('auto_sequence', True)}")
291
292 except Exception as e:
293     # Gestion des erreurs de rechargement
294     print(f"[CONFIG] Erreur rechargement: {e}")
295     return False
296
297 # Indiquer au gestionnaire LED que la séquence est en cours
298 if led_indicator:
299     led_indicator.set_pattern('sequence_running')
300
301 # Valeur de retour agrégée
302 sequence_success = True
303
304 # Récupération des paramètres de simulation via l'API
305 from app.api_client import get_simulation_data
306 try:
307     # Appel API pour récupérer les données de simulation
308     simulation_data = get_simulation_data(mac_address)
309     # Vérification de la validité des données
310     if simulation_data is None:
311         print("[SIMULATION] Erreur: impossible de récupérer les paramètres API")
312         # Indication LED d'erreur API
313         if led_indicator:
314             led_indicator.set_pattern('api_failed')
315         return False
316 except requests.exceptions.ConnectionError:
317     # Gestion des erreurs de connexion Internet
318     print("[SIMULATION] Erreur: Pas de connexion Internet")
319     # Indication LED de problème de connexion
320     if led_indicator:
321         led_indicator.set_pattern('no_internet')
322     return False
323
324 except Exception as e:
325     # Gestion des autres erreurs API
326     print(f"[SIMULATION] Erreur API critique: {e}")
327     # Indication LED d'erreur API
328     if led_indicator:
329         led_indicator.set_pattern('api_failed')
330     return False
331
332 # Vérification des entrées 24V (contacts secs)
333 test_24v_dry_contact()
334
335 # Boucle de traitement des canaux
336 # Compteur de canaux traités
337 channels_processed = 0
338 # Compteur de canaux en erreur
339 channels_failed = 0
340
341 # Boucle sur tous les canaux configurés
342 for entry in cfg["channels"]:
343     # Vérification du flag d'arrêt avant chaque canal
344     if is_adc_stop_requested():
345         print("[SEQUENCER] Arrêt demandé - interruption de la séquence")
346         return False
```

```
345
346 # Numéro du canal à traiter
347 ch = entry["channel"]
348 # Type de capteur configuré
349 sensor_name = entry["sensor"]
350 # État d'activation du canal
351 enabled = entry.get("enabled", True)
352
353 # Ignorer les canaux désactivés
354 if not enabled:
355     print(f"[CONFIG] Canal {ch} désactivé - ignoré")
356     continue
357
358 # Récupération du profil sonde
359 profile = get_profile(sensor_name)
360 # Incrément du compteur de canaux traités
361 channels_processed += 1
362
363 # Affichage du début de traitement du canal
364 print(f"\n[MESURE] Canal {ch} - {sensor_name}")
365 try:
366     # Sélection de la résistance de référence et configuration ADC
367     # Sélection de la résistance de référence via le multiplexeur
368     tmux.select_rref_ohm(profile["rref_ohm"])
369     # Configuration du canal ADC avec les paramètres du profil
370     adc.configure_channel(ch, profile["pga_gain"], profile["idac_uA"])
371     # Lecture et affichage du gain configuré
372     adc.read_gain()
373     # Lecture et affichage de la référence
374     adc.read_ref()
375     # Lecture et affichage du multiplexeur d'entrée
376     adc.read_inpmux()
377
378     # Mesure de la résistance
379     # Mesure de la résistance avec les paramètres du profil
380     r = adc.measure_resistance(profile["rref_ohm"], profile["pga_gain"], cfg["timeout_s"])
381     # Vérification de la validité de la mesure
382     if r is None:
383         print("[MESURE] Résistance: NaN - Erreur de mesure")
384         # Incrément du compteur d'erreurs
385         channels_failed += 1
386         # Affichage d'erreur LED et continuation
387         show_error_and_continue('measure_failed')
388         continue
389     else:
390         # Affichage de la résistance mesurée
391         print(f"[MESURE] Résistance: {r:.6f} ohms")
392
393     # Mesure de la température à partir de l'ADC
394     # Mesure de la température avec conversion selon le type de sonde
395     temperature = adc.measure_temperature(sensor_name, profile["rref_ohm"], profile["pga_gain"], cfg["timeout_s"])
396     # Vérification de la validité de la mesure de température
397     if temperature is None:
398         print("[MESURE] Température non mesurable ou saturation détectée")
399         # Incrément du compteur d'erreurs
400         channels_failed += 1
401         # Affichage d'erreur LED et continuation
402         show_error_and_continue('measure_failed')
403         continue
404     else:
405         # Affichage de la température mesurée
406         print(f"[MESURE] Température mesurée: {temperature:.2f}°C")
407
408     # Sauvegarde locale de l'état
409     save_last_state(channel=ch, resistance_ohm=r, temperature_c=temperature, error=None)
410
411     # Envoi de la mesure vers l'API distante
412     from app.api_client import send_temperature_measurement
413     try:
414         # Tentative d'envoi de la température vers l'API
415         if send_temperature_measurement(mac_address, temperature, ch):
416             print(f"[API] Température {temperature:.2f}°C envoyée avec succès pour le canal {ch}")
417         else:
418             print(f"[API] Erreur lors de l'envoi de la température pour le canal {ch}")
419             # Affichage court d'erreur API
420             show_error_and_continue('api_failed', 1)
421     except requests.exceptions.ConnectionError:
422         # Gestion spécifique des erreurs de connexion
423         print(f"[API] Erreur connexion pour canal {ch}")
424         # Affichage court d'erreur de connexion
425         show_error_and_continue('no_internet', 1)
426     except Exception as e:
427         # Gestion des autres erreurs d'envoi
428         print(f"[API] Erreur envoi canal {ch}: {e}")
429         # Affichage court d'erreur API
430         show_error_and_continue('api_failed', 1)
431
```

```
432         # Calcul et application de la simulation
433         print("[SIMULATION] Calcul et application...")
434
435         # TODO: Vérification de cohérence entre sonde API et sonde canal (temporairement désactivée)
436         # api_probe_type = simulation_data.get("probe_type")
437         # if api_probe_type != sensor_name:
438         #     print(f"[SIMULATION] ERREUR: Incohérence type de sonde!")
439         #     print(f"[SIMULATION] Sonde API: '{api_probe_type}' vs Sonde Canal {ch}: '{sensor_name}'")
440         #     print(f"[SIMULATION] La simulation est ignorée pour ce canal")
441         #     channels_failed += 1
442         #     show_error_and_continue('config_error')
443         #     continue
444
445         # Injection de la température mesurée dans les données de simulation
446         simulation_data["temperature"] = temperature
447         # Injection du type de sonde (temporaire - remplace le type API)
448         simulation_data["probe_type"] = sensor_name
449         # Affichage des paramètres de simulation
450         print(f"[SIMULATION] Paramètres: n={simulation_data['n']}, k_m={simulation_data['k_m']}")
451         # Affichage des températures utilisées
452         print(f"[SIMULATION] T_mes={temperature:.2f}°C, T_prev={simulation_data['forecast_temperature']:.2f}°C")
453
454         # Calcul de la température simulée
455         t_sim = run_temperature_simulation(simulation_data)
456         # Vérification de la validité du calcul
457         if t_sim is None:
458             print("[SIMULATION] Erreur lors du calcul de T_sim")
459             # Incrément du compteur d'erreurs
460             channels_failed += 1
461             # Affichage d'erreur LED et continuation
462             show_error_and_continue('measure_failed')
463             continue
464         # Affichage de la température simulée
465         print(f"[SIMULATION] Température simulée: {t_sim:.2f}°C")
466
467         # Mise à jour de l'état avec la température simulée
468         save_last_state(channel=ch, resistance_ohm=r, temperature_c=temperature, temperature_sim_c=t_sim, error=None)
469
470         # Conversion de la température simulée en résistance cible
471         resistance_target = convert_temperature_to_resistance(t_sim, sensor_name)
472         # Vérification de la validité de la conversion
473         if resistance_target is None:
474             print(f"[SIMULATION] Erreur conversion T_sim → résistance pour {sensor_name}")
475             # Incrément du compteur d'erreurs
476             channels_failed += 1
477             # Affichage d'erreur LED et continuation
478             show_error_and_continue('measure_failed')
479             continue
480         # Affichage de la résistance cible calculée
481         print(f"[SIMULATION] Résistance cible: {resistance_target:.2f} ohms")
482
483         # Application de la résistance simulée via le multiplexeur
484         resistance_sim = ResistanceSimulator()
485         # Tentative d'application de la résistance via le MUX
486         if resistance_sim.apply_resistance_simulation(resistance_target, ch, sensor_name):
487             print(f"[SIMULATION] Résistance appliquée via MUX sur canal {ch} (sonde: {sensor_name})")
488         else:
489             print(f"[SIMULATION] Erreur lors de l'application MUX sur canal {ch}")
490             # Incrément du compteur d'erreurs
491             channels_failed += 1
492             # Affichage d'erreur LED et continuation
493             show_error_and_continue('measure_failed')
494
495         # Pause entre canaux
496         time.sleep(cfg["inter_measure_sleep_s"])
497
498     except Exception as e:
499         # Gestion des erreurs générales de traitement du canal
500         print(f"[ERREUR] Canal {ch}: {e}")
501         # Incrément du compteur d'erreurs
502         channels_failed += 1
503         # Affichage d'erreur critique LED et continuation
504         show_error_and_continue('critical_error')
505         continue
506
507 # Évaluation finale de la séquence
508 # Cas où aucun canal n'a échoué
509 if channels_failed == 0:
510     print(f"[SEQUENCER] Séquence réussie - {channels_processed} canaux traités")
511     # Motif LED normal après succès
512     if led_indicator:
513         # Motif LED normal après succès
514         led_indicator.set_pattern('normal')
515     sequence_success = True
516 # Cas où certains canaux ont échoué mais pas tous
517 elif channels_failed < channels_processed:
518     print(f"[SEQUENCER] Séquence partielle - {channels_processed - channels_failed}/{channels_processed} canaux réussis")
```

```
519         # Indication LED d'erreur de mesure partielle
520         if led_indicator:
521             led_indicator.set_pattern('measure_failed')
522             sequence_success = True
523     # Cas où tous les canaux ont échoué
524     else:
525         print(f"[SEQUENCER] Séquence échouée - Tous les canaux ont échoué")
526         # Indication LED d'erreur critique
527         if led_indicator:
528             led_indicator.set_pattern('critical_error')
529             sequence_success = False
530
531     # Affichage de fin de séquence avec horodatage
532     print(f"[SEQUENCER] Séquence terminée - {datetime.now().strftime('%H:%M:%S')}")
533     return sequence_success
534
535
536 # -----
537 # Fonctions utilitaires de rechargement dynamique
538 # -----
539
540 def reload_timing_config():
541     """
542     @brief   Recharge les paramètres de temporisation depuis le fichier de configuration.
543     @details Lit uniquement les paramètres de temporisation pour mise à jour dynamique
544             sans perturber le reste du système. Utilisé pour permettre la modification
545             des intervalles de séquence et du mode automatique via l'interface web.
546
547     @return  Tuple (auto_sequence, sequence_interval_minutes, success)
548             où success indique si le rechargement a réussi.
549     """
550     try:
551         # Chemin vers le fichier de configuration
552         CONFIG_FILE = os.path.join(SRC_DIR, "config_module", "sensors.json")
553         # Rechargement de la configuration complète
554         cfg = load_config(CONFIG_FILE)
555
556         # Extraction des paramètres de temporisation
557         auto_sequence = cfg.get("auto_sequence", True)
558         sequence_interval_minutes = cfg.get("sequence_interval_minutes", 5)
559
560         # Log des paramètres rechargés
561         print(f"[CONFIG] Paramètres rechargés - Auto: {auto_sequence}, Intervalle: {sequence_interval_minutes}min")
562
563         return auto_sequence, sequence_interval_minutes, True
564     except Exception as e:
565         # Log d'erreur en cas d'échec
566         print(f"[CONFIG] Erreur rechargement paramètres: {e}")
567         # Retour avec valeurs par défaut
568         return True, 5, False
569
570
571 # -----
572 # Point d'entrée principal du programme
573 # -----
574
575 def main():
576     """
577     @brief   Point d'entrée principal du programme.
578     @details Parse les arguments de la ligne de commande, initialise le
579             matériel (GPIO, ADC, MUX), le gestionnaire LED, et exécute le
580             séquenceur en mode automatique ou manuel selon la configuration.
581     @return  None
582     """
583     global sequencer_running, led_indicator
584
585     # Analyse des options en ligne de commande
586     # Création du parser d'arguments
587     parser = argparse.ArgumentParser(description="Système de mesure et simulation de température")
588     # Option pour le mode manuel
589     parser.add_argument("--manual", action="store_true", help="Mode manuel (demande confirmation à chaque séquence)")
590     # Option pour une exécution unique
591     parser.add_argument("--once", action="store_true", help="Exécute une seule séquence puis s'arrête")
592     # Option pour le serveur web
593     parser.add_argument("--webui", action="store_true", help="Lance le serveur web sur le port 8080")
594     # Parsing des arguments fournis
595     args = parser.parse_args()
596
597     # Affichage du titre du programme
598     print("=== SYSTÈME DE MESURE ET SIMULATION ===")
599
600     # Chargement de la configuration
601     CONFIG_FILE = os.path.join(SRC_DIR, "config_module", "sensors.json")
602     # Chargement initial de la configuration
603     cfg = load_config(CONFIG_FILE)
604
605     # Paramètres de séquence automatique
```

```
606 # Récupération du mode automatique depuis la config
607 auto_sequence = cfg.get("auto_sequence", True)
608 # Récupération de l'intervalle entre séquences
609 sequence_interval_minutes = cfg.get("sequence_interval_minutes", 5)
610
611 # Présentation des canaux configurés
612 # Filtrage des canaux activés
613 enabled_channels = [entry for entry in cfg["channels"] if entry.get("enabled", True)]
614 # Filtrage des canaux désactivés
615 disabled_channels = [entry for entry in cfg["channels"] if not entry.get("enabled", True)]
616
617 # Affichage du nombre de canaux activés
618 print(f"[CONFIG] Canaux activés: {len(enabled_channels)}")
619 # Affichage du détail des canaux activés
620 for entry in enabled_channels:
621     print(f"[CONFIG] Canal {entry['channel']}: {entry['sensor']}")
622
623 # Affichage des canaux désactivés s'il y en a
624 if disabled_channels:
625     print(f"[CONFIG] Canaux désactivés: {len(disabled_channels)}")
626     # Affichage du détail des canaux désactivés
627     for entry in disabled_channels:
628         print(f"[CONFIG] Canal {entry['channel']}: {entry['sensor']} (OFF)")
629
630 # Affichage du mode choisi
631 # Mode manuel demandé en ligne de commande
632 if args.manual:
633     print(f"[CONFIG] Mode: MANUEL (demande confirmation)")
634 # Mode exécution unique
635 elif args.once:
636     print(f"[CONFIG] Mode: UNE SEULE FOIS")
637 # Mode automatique configuré
638 elif auto_sequence:
639     print(f"[CONFIG] Mode: AUTOMATIQUE (toutes les {sequence_interval_minutes} minutes)")
640     print(f"[CONFIG] Commandes disponibles pendant l'exécution:")
641     print(f"[CONFIG] - Appuyez sur ENTRÉE ou tapez 's' pour forcer une séquence")
642     print(f"[CONFIG] - Tapez 'q' pour quitter")
643 # Mode manuel par configuration
644 else:
645     print(f"[CONFIG] Mode: MANUEL (auto_sequence=false dans config)")
646
647 # Démarrage optionnel du serveur web en arrière-plan
648 webui_thread = None
649 # Vérification si le serveur web est demandé
650 if args.webui:
651     print(f"[WEBUI] Démarrage du serveur web sur http://localhost:8080")
652     # Création du thread pour le serveur web
653     webui_thread = threading.Thread(target=start_webui_server, daemon=True)
654     # Démarrage du thread serveur web
655     webui_thread.start()
656     # Laisser le temps au serveur de monter
657     time.sleep(2)
658
659 # Affichage des motifs LED disponibles (référence)
660 print(f"\n[LED] Patterns d'erreur disponibles:")
661 print(f"[LED] - NORMAL: LED allumée en continu")
662 print(f"[LED] - SEQUENCE: Clignotement continu pendant mesure")
663 print(f"[LED] - NO_INTERNET: 1 clignotement sur 10s (pas de connexion Internet)")
664 print(f"[LED] - API_FAILED: 2 clignotements sur 10s (erreur API/connexion)")
665 print(f"[LED] - MEASURE_FAILED: 3 clignotements sur 10s (erreur mesure ADC/résistance)")
666 print(f"[LED] - CRITICAL_ERROR: 4 clignotements sur 10s (erreur critique système)")
667 print(f"[LED] - CONFIG_ERROR: 5 clignotements sur 10s (erreur de configuration)")
668
669 # Ligne vide pour la lisibilité
670 print()
671
672 # Installation du handler pour Ctrl+C
673 signal.signal(signal.SIGINT, signal_handler)
674
675 # Remise à zéro du flag d'arrêt ADC au démarrage
676 reset_adc_stop_flag()
677 print("[ADC] Système d'interruption ADC initialisé")
678
679 # Initialisation du gestionnaire LED
680 # Création de l'instance de gestion LED
681 led_indicator = create_led_indicator(GPIO_CHIP_PATH, FRONT_LED)
682 # Vérification de l'initialisation LED
683 if led_indicator:
684     print("[LED] Système LED initialisé - LED normale ON")
685 else:
686     print("[LED] Erreur : Impossible d'initialiser la LED")
687
688 # Initialisation du matériel GPIO/ADC/MUX
689 # Initialisation du relais de commande principal
690 relay = GPIO(GPIO_CHIP_PATH, CMD_RELAY, "out")
691 # Activer le relais principal
692 relay.write(True)
```



```
693 # Instance du multiplexeur
694 tmux = Tmux1204()
695 # Instance de l'ADC
696 adc = Ads124s08()
697 # Récupération de l'adresse MAC depuis la configuration
698 mac_address = cfg.get("mac_address", "0030DEABCDEF")
699
700 try:
701     # Activation du séquenceur principal
702     sequencer_running = True
703     # Calcul du temps de la première séquence
704     next_sequence_time = datetime.now()
705
706     # Thread d'écoute utilisateur pour le mode automatique
707     input_thread = None
708     # Vérification si le thread d'écoute est nécessaire
709     if auto_sequence and not args.manual and not args.once:
710         # Création du thread de surveillance des entrées
711         input_thread = threading.Thread(target=input_monitor, daemon=True)
712         # Démarrage du thread de surveillance
713         input_thread.start()
714         print("[MANUAL] Thread de surveillance des entrées démarré")
715
716     # Boucle principale du séquenceur
717     # Boucle tant que le séquenceur est actif et pas d'arrêt demandé
718     while sequencer_running and not stop_event.is_set():
719         # Récupération du temps actuel
720         current_time = datetime.now()
721
722         # Initialisation des variables de contrôle
723         should_run = False
724         sequence_reason = ""
725
726         # Déclenchement manuel depuis input_monitor
727         # Vérification d'un déclenchement manuel
728         if force_sequence.is_set():
729             should_run = True
730             sequence_reason = "MANUELLE"
731             # Remise à zéro du flag de déclenchement manuel
732             force_sequence.clear()
733             # Rechargement des paramètres avant reprogrammation du cycle
734             auto_sequence_updated, sequence_interval_minutes_updated, reload_success = reload_timing_config()
735             if reload_success:
736                 auto_sequence = auto_sequence_updated
737                 sequence_interval_minutes = sequence_interval_minutes_updated
738             # Reprogrammation du cycle automatique si applicable
739             if auto_sequence and not args.manual:
740                 # Recalcule le temps de prochaine séquence avec le nouvel intervalle
741                 next_sequence_time = current_time + timedelta(minutes=sequence_interval_minutes)
742                 print(f"[SEQUENCER] Cycle automatique redémarré - Prochaine séquence à: {next_sequence_time.strftime('%H:%M:%S')}")
743             # Mode exécution unique
744             elif args.once:
745                 should_run = True
746                 sequence_reason = "UNIQUE"
747                 # Arrêt après une exécution
748                 sequencer_running = False
749             # Mode manuel interactif
750             elif args.manual or not auto_sequence:
751                 print(f"[SEQUENCER] Appuyez sur Entrée pour lancer une séquence (Ctrl+C pour arrêter)...")
752                 try:
753                     # Attente d'une entrée utilisateur
754                     input()
755                     should_run = True
756                     sequence_reason = "MANUELLE"
757                 except EOFError:
758                     break
759             # Mode automatique basé sur l'horloge
760             else:
761                 # Vérification si c'est l'heure d'une nouvelle séquence
762                 if current_time >= next_sequence_time:
763                     should_run = True
764                     sequence_reason = "AUTOMATIQUE"
765                     # Programmation de la prochaine séquence
766                     next_sequence_time = current_time + timedelta(minutes=sequence_interval_minutes)
767                     print(f"[SEQUENCER] Prochaine séquence automatique programmée à: {next_sequence_time.strftime('%H:%M:%S')}")
768
769         # Exécution de la séquence si déclenchée
770         if should_run:
771             # Rechargement des paramètres de temporisation avant séquence (si pas déjà fait)
772             if sequence_reason != "MANUELLE": # Pour les manuelles, déjà fait plus haut
773                 print(f"[SEQUENCER] Rechargement paramètres temporisation...")
774                 auto_sequence_updated, sequence_interval_minutes_updated, reload_success = reload_timing_config()
775                 if reload_success:
776                     # Sauvegarde de l'ancien intervalle pour comparaison
777                     old_interval = sequence_interval_minutes
778                     # Mise à jour des paramètres pour les prochains cycles
779                     auto_sequence = auto_sequence_updated
```

```

780         sequence_interval_minutes = sequence_interval_minutes_updated
781         # Recalcul du prochain cycle si en mode automatique et intervalle modifié
782         if sequence_reason == "AUTOMATIQUE" and sequence_interval_minutes_updated != old_interval:
783             next_sequence_time = current_time + timedelta(minutes=sequence_interval_minutes)
784             print(f"[SEQUENCER] Intervalle mis à jour: {old_interval}min → {sequence_interval_minutes}min")
785
786         print(f"[SEQUENCER] Démarrage séquence {sequence_reason} - {current_time.strftime('%H:%M:%S')}")
787         # Appel de la fonction de séquence de mesure
788         success = run_measurement_sequence(cfg, mac_address, adc, tmux)
789         # Vérification du succès de la séquence
790         if success:
791             print(f"[SEQUENCER] Séquence {sequence_reason} terminée avec succès")
792         else:
793             print("[SEQUENCER] Erreur lors de la séquence")
794
795         # Pause courte pour permettre la réactivité aux entrées
796         # Pause seulement en mode automatique continu
797         if auto_sequence and not args.manual and not args.once and sequencer_running:
798             time.sleep(1)
799
800     except KeyboardInterrupt:
801         # Gestion de l'interruption par Ctrl+C
802         print("\n[SEQUENCER] Arrêt demandé par l'utilisateur...")
803     finally:
804         # Arrêt du séquenceur
805         sequencer_running = False
806         # Déclenchement de l'événement d'arrêt
807         stop_event.set()
808
809         # Arrêt propre du thread d'entrée si actif
810         # Vérification de l'existence et de l'activité du thread
811         if input_thread and input_thread.is_alive():
812             print("[MANUAL] Arrêt du thread de surveillance...")
813             # Attente de la fin du thread avec timeout
814             input_thread.join(timeout=1)
815
816         # Tentatives de fermeture des périphériques matériels
817         # Fermeture de l'ADC
818         try:
819             adc.close()
820         except Exception:
821             pass
822         # Fermeture du multiplexeur
823         try:
824             tmux.close()
825         except Exception:
826             pass
827         # Fermeture du gestionnaire LED
828         try:
829             if led_indicator:
830                 led_indicator.close()
831         except Exception:
832             pass
833         # Désactivation et fermeture du relais
834         try:
835             relay.write(False)
836             relay.close()
837         except Exception:
838             pass
839
840         # Message final de confirmation d'arrêt
841         print("[SEQUENCER] Système arrêté proprement")
842
843 # -----
844 # Point d'entrée principal
845 # -----
846
847 if __name__ == "__main__":
848     main()
849

```